Full-Stack Engineering Training — Day 12 Study Material

MODULE	TOPIC	PORT & STACK
Backend Validation Architecture	DTOs, class-validator & Error Handling	Port 3012 • ValidationPipe

Key Learning Objectives: Understand the critical role of Data Transfer Objects (DTOs) in modern enterprise APIs, declarative validation using `class-validator` and `class-transformer`, mass assignment security protection with `whitelist: true` and `forbidNonWhitelisted: true`, nested DTO validation with `@ValidateNested()`, and standardizing application error formats with custom `AppException` and global `AllExceptionsFilter`.

1. Mental Model: Why DTOs and Validation Pipes Matter?

APIs must never trust client payloads. Unvalidated inputs lead to SQL crashes, runtime `TypeError` exceptions, and severe mass-assignment security vulnerabilities where users inject unauthorized fields like `"role": "ADMIN"`.

DTO vs Database Entity:
- **DTO**: Defines what the client is permitted to send across the network. Handles input sanitation and validation.
- **Entity**: Defines how records are structured and persisted in PostgreSQL.

2. Declarative Validation with class-validator

Instead of manual validation checks, NestJS evaluates declarative TypeScript decorators before route handlers execute:

Decorator	Example Usage	Validation Rule
`@IsString()`	`@IsString()`	Must be a primitive string
`@MinLength()`	`@MinLength(3)`	Enforces minimum string character count
`@MaxLength()`	`@MaxLength(100)`	Prevents payload overflow beyond 100 characters
`@IsEnum()`	`@IsEnum(ProjectStatus)`	Matches values strictly against predefined enum constants
`@IsInt()`	`@IsInt()`	Ensures number has no fractional component
`@Min() / @Max()`	`@Min(1) @Max(50)`	Enforces lower and upper numeric boundaries
`@Transform()`	`@Transform(({value}) => value.trim())`	Sanitizes values prior to validation (strips leading/trailing spaces)

3. Validation Pipe & Mass Assignment Protection

Global registration in `main.ts` ensures that no unvalidated or unauthorized field reaches any service:

```
// main.ts
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,            // Strips any property lacking a DTO decorator
    forbidNonWhitelisted: true, // Rejects requests containing unrecognized properties
    transform: true             // Automatically transforms request body into DTO instances
  })
);
```

Rejection of Malicious Payloads:

If an unauthorized payload is submitted:

```
// Incoming POST /projects
{
  "name": "Apex Commercial Park",
  "status": "ACTIVE",
  "towerCount": 4,
  "role": "SUPERADMIN" // <-- Attack injection!
}

// Resulting 400 Bad Request Response
{
  "statusCode": 400,
  "message": ["property role should not exist"],
  "error": "Bad Request",
  "timestamp": "2026-10-14T02:00:00.000Z",
  "path": "/projects"
}
```

4. Nested DTO Validation (@ValidateNested + @Type)

When validating nested object hierarchies (such as a Tower containing an array of Floor objects), both `@ValidateNested()` and `@Type()` must be provided:

```
// towers/dto/floor.dto.ts
export class FloorDto {
  @IsInt()
  @Min(1)
  floorNumber: number;

  @IsEnum(['NOT_STARTED', 'IN_PROGRESS', 'COMPLETED'])
  status: string;
}

// towers/dto/tower.dto.ts
export class TowerDto {
  @IsString()
  @MinLength(2)
  name: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => FloorDto) // Required to reconstruct class instance at runtime
  floors: FloorDto[];
}
```

5. Global Exception Filter & Consistent Error Schema

Enterprise frontends need consistent error structures so they can handle failures reliably. Our global `AllExceptionsFilter` wraps every exception into a standardized 5-field JSON response:

```
// common/filters/http-exception.filter.ts
{
  "statusCode": 404,
  "message": "Project with ID proj-999 was not found",
  "error": "NotFoundException",
  "timestamp": "2026-10-14T02:00:00.000Z",
  "path": "/projects/proj-999"
}
```

6. Common Beginner Pitfalls & "Intern Watch-Outs"

> **1. Forgetting @Type() with @ValidateNested():**
> TypeScript types are erased at runtime. Without `@Type(() => FloorDto)`, `class-validator` sees a plain generic object and skips all nested child validation!

> **2. Not Setting forbidNonWhitelisted: true:**
> Setting only `whitelist: true` silently strips unknown fields without returning an error. Enabling `forbidNonWhitelisted: true` ensures API clients are immediately alerted if they send invalid field names.

> **3. Missing transform: true:**
> Without `transform: true`, query strings like `?towerCount=5` remain strings and fail numeric validators like `@IsInt()`.

7. Summary & Memory Cheat Sheet

- **The Twin Decorators**: Always pair `@ValidateNested()` with `@Type(() => ChildDto)`.
- **Security Pipeline**: Whitelist → Forbid Non-whitelisted → Transform → Execute Handler.
- **5-Field Error Schema**: `statusCode`, `message`, `error`, `timestamp`, `path`.

8. Verification & Exercise Output (o/p)

```
--- RUNNING DAY 12 VALIDATION & ERROR HANDLING TESTS ---
✓ Test 1: Valid project created with trimmed name and validated fields (201)
✓ Test 2: Mass assignment / non-whitelisted field rejected (400 Bad Request)
✓ Test 3: Short name < 3 chars rejected with descriptive message (400)
✓ Test 4: Invalid enum status rejected with allowed enum choices (400)
✓ Test 5: Tower count upper bound > 50 rejected (400)
✓ Test 6: Global Exception Filter returned consistent 5-field schema (404)
✓ Test 7: Nested TowerDto and FloorDto created successfully (201)
✓ Test 8: Nested validation correctly caught error inside child FloorDto array (400)
ALL 8 VALIDATION TESTS PASSED CLEANLY!
```